Convolutional Neural Networks (CNN)

Convolutional Neural Networks (CNNs) are a type of Deep Learning neural network specially designed for processing grid-like data, such as images and videos. CNNs automatically learn important features like edges, textures, shapes, and objects from images.

Why Can't We Use ANN for Images?

Although ANN can process images, it is not practical because of the following problems:

1. High Computational Power

ANN connects every pixel with every neuron, resulting in a huge number of parameters.

2. Spatial Arrangement is Missing

ANN treats every pixel independently and cannot understand the relationship between neighboring pixels.

3. Overfitting

Because ANN has too many parameters for image data, it easily memorizes training images instead of learning useful patterns.

What Actually is CNN?

CNN processes an image differently from ANN.

Instead of looking at every pixel at once, CNN looks at **small patches** of the image using filters and gradually learns more complex features.

ANN:

Looks at every pixel individually.

CNN:

Looks at small regions (patches) and learns features step by step.

CNN Architecture (Theory)

The basic architecture of CNN consists of the following layers:

Input Image

↓

Convolution Layer

↓

Pooling Layer

↓

Convolution Layer

↓

Pooling Layer

↓

Flatten Layer

↓

Fully Connected Layer (ANN)

↓

Output

1. Convolution Layer (The Detective 🔍)

The **Convolution Layer** is the feature extraction layer of CNN. It uses small filters (kernels) that slide over the image to detect important patterns.

Logic

A small magnifying glass (3×3 filter) moves across the picture.

The filter may highlight:

- Edges
- Colors
- Textures

Example:

One filter learns straight lines.

Another filter learns round shapes.

Feature Extraction

CNN learns features layer by layer.

First Layer

Detects primitive features like:

- Edges
- Lines
- Corners

Middle Layers

Detect:

- Eyes
- Nose
- Mouth
- Wheels
- Hair

Deep Layers

Detect complete objects like:

- Cat
- Car
- Castle

Types of Images

There are two types of images:

1. Grayscale Images

Pixel values range from:

0 → Black

255 → White

2. RGB Images

RGB images contain three channels:

- Red
- Green
- Blue

Each channel contains values from **0–255**.

Since RGB contains three stacked matrices, it forms a **3D Tensor (Cuboid)** instead of a single matrix.

Filters (Kernels)

CNN uses filters for edge detection.

Horizontal Filter

-1 -1 -1

0 0 0

1 1 1

Vertical Filter

-1 0 1

-1 0 1

-1 0 1

Logic

We place the filter over the image matrix.

↓

Multiply corresponding values.

↓

Take the sum.

↓

Move the filter.

↓

A new matrix called **Feature Map** is generated.

For RGB images, convolution is performed on the tensor (cuboid).

Padding

Padding means adding extra pixels (usually zeros) around the image before applying convolution.

Why Padding?

Without padding:

- Corner pixels are used less.
- Image size decreases after convolution.
- Important information near image borders may be lost.

Example:

5×5 image

↓

3×3 filter

↓

Output becomes 3×3

To preserve image size, we add a border of zeros.

0 0 0 0 0 0 0

0 0

0 0

0 0

0 0

0 0

0 0 0 0 0 0 0

Strides

Stride is the number of pixels the filter moves after each convolution operation.

Logic

Stride determines how much the filter moves forward at one time.

Larger stride:

- Reduces image size
- Reduces computation
- May lose image information

Example:

5×5 image

↓

Stride = 2

↓

Output becomes approximately 2×2

Low-Level Features

Large strides capture fewer details.

Nowadays, large strides are less common because modern computers have much higher computational power.

Pooling Layer

The **Pooling Layer** reduces the spatial size of feature maps while preserving the most important information.

Why Pooling?

As feature maps increase,

memory and computation also increase.

Pooling reduces:

- Image size
- Parameters
- Computation
- Overfitting

Parameters:

Pool Size = (2,2)

Stride = 2

Type = Max Pooling

Types of Pooling

1. Max Pooling

Selects the maximum value from each pooling window

Logic

Captures dominant features.

Keeps strong information.

ReLU is usually applied so:

Negative values → 0

Positive values remain unchanged.

2. Average Pooling

Calculates the average value from each pooling window.

Logic

Captures average features.

Useful when smoother, less noisy features are required.

3. Minimum Pooling

Selects the minimum value from each pooling window.

Logic

Captures weak features.

Less commonly used.

Flatten Layer

The **Flatten Layer** converts the 2D (or 3D feature maps) into a 1D vector so it can be given as input to an Artificial Neural Network.

Example

Input Image

28 × 28 × 1

↓

Convolution

32 filters (3×3)

↓

Feature Maps

26 × 26 × 32

↓

Pooling (2×2)

↓

13 × 13 × 32

↓

Second Convolution

64 filters

↓

11 × 11 × 64

↓

Pooling

↓

5 × 5 × 64

↓

Flatten

↓

1600 neurons

↓

ANN

↓

ReLU

↓

Gradient Descent

↓

Loss Function

↓

Optimizer

↓

Prediction

Fully Connected Layer (Decision Maker)

The **Fully Connected Layer** works exactly like an ANN. It receives the flattened features and performs classification.

Example

70% → Cat

20% → Dog

10% → Rabbit

Complete CNN Workflow

Input Image

↓

Convolution Layer (Feature Extraction)

↓

ReLU Activation

↓

Pooling Layer

↓

Convolution Layer

↓

ReLU

↓

Pooling Layer

↓

Flatten Layer

↓

Fully Connected Layer (ANN)

↓

Loss Function

↓

Backpropagation

↓

Optimizer

↓

Final Prediction

Final Connection

Images

↓

ANN is computationally expensive and ignores spatial information

↓

CNN solves these problems using Filters

↓

Convolution extracts features

↓

Pooling reduces dimensions

↓

Flatten converts features into a vector

↓

Fully Connected Layer performs classification

↓

Final Output (Cat, Dog, Car, etc.)